

# **Computer Security - CheatSheet**

IN BA5 - Thomas Bourgeat

Notes by Ali EL AZDI

October 26th, 2025



**CompSec Properties - Confidentiality.** prevent unauthorized disclosure of information. *authorized users may read a file*

- **Integrity.** prevent unauthorized modification of information. *authorized programs may write a file*

- **Availability.** prevent unauthorized denial of service or access to information and resources. *authorized services can access a file*

- **Authenticity.** assurance that entities (users, systems, or data) are genuine and can be verified as such.

- **Anonymity.** protection of an individual's identity from being disclosed or linked to specific actions or data.

- **Non-repudiation.** assurance that a party in a communication cannot deny the authenticity of their signature or the sending of a message.

**The Adversary.** malicious entity aiming at breaching the security policy and **will** choose the optimal way to use its ressources to mount an attack that violates the security properties.

**Threat Model.** describes the ressources available to the adversary and their capabilities *(has access to internet, but doesn't have access to the internal network of the company.)*

**Threat.** Who might attack which assets, using what resources, with what goal, how, and with what probability

**Vulnerability.** Specific weakness that adversaries could exploit with interest in a lot of assets *(API is not protected, password appears in plain text...)*

Principles of CompSec.

- 1. **Economy of mechanism** Keep security mechanism design simple and small ⇒ Easier to audit and verify, testing is **not** appropriate to evaluate security.
- **Trusted Computing Base (TCB).** Every component of the system on which the security policy relies upon *hardware, firmware, software.*  
The TCB is trusted to operate correctly for the security policy to hold. → If something goes wrong in it, the security policy may be violated
- must** be kept small to easy verification / diminish the attack surface
- 2. **Fail-safe defaults.** Base access decisions on permission rather than exclusion. *(Whitelist, do not blacklist)*  
If something fails, be as secure as it does not fail errors / uncertainty should error on the side of the security policy.
- 3. **Complete mediation..** Every access to every object must be checked for authority. A **Reference Monitor** mediates all actions from subjects on objects and ensures they are according to the policy. *△time to check vs. time to use*
- 4. **Open design** The design should not be secret *Always design as if the enemy knows the system. (Crypto-only secret is key.Authentication-Only secret is password.for Obfuscation-Only used secret is noise.).assuming the thread model can't get a hold of the system is unrealistic (employee corruption, ...)*

**Access Control.** Security mechanism that ensures all accesses and actions on objects by principals are within the security policy.

**no chicken soup** *(checks everywhere in code), use a reference monitor, module used all over code checking for subjects and actions*

**Discretionary Access Control (DAC).** Object owners assign permissions, ownership of resources. *Linux, Social Networks*

**Mandatory Access Control (MAC).** Central security policy assigns permissions, usually for organizations with need for central control. *Military, Hospital, ...*

**Access Control Matrix.** abstract representation of all permitted triplets of (subject, object, access right) within a system.

|       | file1       | file2       | file3       |
|-------|-------------|-------------|-------------|
| Alice | read, write |             | read        |
| Bob   |             | read, write | read, write |

*Complexity. O(f · u)*

**Access Control List (ACL).** associate permissions to objects, stores permissions close to the resource.

*file1: (Alice,read/write), file2: (Bob, read/write), file3: (Alice,read),(Bob,read/write)*

⊗ easy to determine who can access a resource and to revoke rights by resource.  
⊗ hard to check all users rights, to remove all perms. for a user, delegate perms.

**Role Based Access Control (RBAC).** access granted based on user roles and predefined permissions. systems have too many subjects (that come and go) → large dynamic ACLs. Subjects are often similar to each other and get assigned the same rights.

- 1. Assign permissions to roles,
- 2. Assign roles to subjects

3. Subjects have the permissions of their assigned role

⊗ role explosion (temptation to create fine grained roles), limited expressiveness, difficult to implement separation of privilege + least privilege.

**Group Based Access Control (GBAC).** access granted based on user group membership. **exactly like RBAC but instead of roles, permissions are assigned to groups.** groups are broader, represent organizational units instead than specific functions.

- 1. Assign permissions to groups
- 2. Assign subjects to groups

3. Subjects have the combined permissions of all their groups

⊗ coarse granularity, overlapping group memberships, inconsistent permissions, difficult to manage if users belong to many groups.

*In case of Negative Permissions check negative permissions first before group, △system crashes before negative checks.*

**Ambient Authority.** an action succeeds if the subject only specifies the *operation* and the *object name*, not the specific authority used.

⊗ leads to accidental misuse of authority, programs may act with more rights than intended. → Confused Deputy Problem.

**Confused Deputy Problem.** when a program (deputy) is tricked into misusing its authority on behalf of another subject.

*Alice runs compiler(input, bill) ⇒ compiler (with write access to bill) overwrites billing file. ⇒ Alice uses compiler's authority to modify bill indirectly.*

⊗ ambient authority allows unintended privilege use.

**Solutions:**

- 1. Restrict privileged process access.
- 2. Make privileged process check user's authorization.
- 3. Use **Capabilities** to explicitly delegate rights.

**Harm.** The bad thing that could happen when the **threat** materialsizes. *(adversary steals the money, learns my password...)*

**Security Policy.** high level description of the security properties that must hold in the system in relation to assets and principals.

- **Assets (objects).** anything with value (data, files, memory) needing protection.

- **Principals (subjects).** people, computer programs, services.

**Security Mechanism.** Technical mechanism used to ensure that the security policy is not violated by an adversary within the threat model, **we can only prepare for threats we're aware of**

*(Policy. ensure messages cannot be read by anyone but the sender and the receiver, Mechanism. encrypt the message before sending)*

**Composition of Security Mechanisms**

- **Defence in depth.** As long as one remains unbroken the Security Policy isn't broken) *(two-factor auth)*
- **Weakest Link.** if anyone fails the Security Policy, it is broken. *(security questions for a lost password → just need to know the answer...)*

*Humans can be vulnerabilities - phishing attacks, bad use of passwords...)*

**To show a system is secure. (under a specific threat model)**

- Attacker - Just one way to violate **one** security property is enough.
- Defender - No adversary strategy can violate the security policy.

**Security Argument.** Rigorous argument that security mechanisms in place are effective in maintaining security policy subject to assumptions on Threat Model.

- 5. **Separation of privilege.** No single accident, deception, or breach of trust is sufficient to compromise the protected information
- **Privilege.** A privilege allows a user to perform an action on a computer system that may have security consequences. *(create a file in a directory...)*
- 6. **Least Privilege.** Every program and every user of the system should operate using the least set of privileges necessary to complete the job. *Rights are added on need, discarded after use. Users should only know about things if they have to.*
- 7. **Least Common Mechanism** Minimize the amount of mechanism common to more than one user and depended on by all users. Every shared mechanism represents a potential information path between users.
- 8. **Psychological acceptability.** It is essential that the human interface be designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly. *(hide complexity, cultural acceptability...)*
- 9. **Work Factor**  
Compare the cost of breaking the mechanism with the resources of a potential attacker. *(cost of compromising insiders, cost of finding a bug, monetization...)*
- 10. **Compromise recording** Detect and record security breaches with tamper-evident logs, ensuring traceability, integrity, confidentiality, and availability of recorded data.

**User & Group Identities.** Most modern systems rely on DAC.

**UIDs / GIDs.** numerical identifiers for users and groups.

*/etc/passwd:* username:password:UID:GID:info:home:shell

*/etc/group:* defines secondary groups.

Each user has a home directory and belongs to one or more groups.

**UNIX Model.** Everything is a file.

| directories | owner       | group | others   | owner    | group |               |            |  |  |  |  |  |  |  |  |  |  |  |  |
|-------------|-------------|-------|----------|----------|-------|---------------|------------|--|--|--|--|--|--|--|--|--|--|--|--|
|             | drwxrwxr-x  | 1     | catronco | catronco | 4096  | Sep 16 14:23  | exampledir |  |  |  |  |  |  |  |  |  |  |  |  |
|             | -rwxrwxr-w- | 1     | catronco | catronco | 8600  | Sep 15 15:20  | hello      |  |  |  |  |  |  |  |  |  |  |  |  |
|             | -rw-rw-rw-  | 1     | catronco | catronco | 150   | Sep 15 15:14  | hello.c    |  |  |  |  |  |  |  |  |  |  |  |  |
| files       | -rw-rw-rw-  | 1     | catronco | catronco | 45    | Sep 15 15:07  | test1.txt  |  |  |  |  |  |  |  |  |  |  |  |  |
|             |             | links |          |          | size  | last modified | filename   |  |  |  |  |  |  |  |  |  |  |  |  |

**Directories.** Read → list files; Write → add/remove files; Exec → traverse (cd).

**Permission check order:** 1. If process UID = file owner → check owner bits.  
2. Else if GID matches → check group bits.  
3. Else → check “other” bits.

*Root (UID 0. bypasses all checks.*

**Changing Permissions.**

**chmod.** modify permission bits. *chmod +r file, chmod 666 file.*

**chown.** change file owner/group(opt.) *chown root:staff /srv/config*

**chgrp.** change file group. *chgrp www-data /var/www*

**Special Rights.**

**suid (set user ID).** run program with owner's privileges. *puts s in owner's x field*  
*chmod u+s filename*  
*allows normal users to change passwords without full root access.*

**sgid (set group ID).** run program with group's privileges.  
*on a directory - creating a file inside gets folder's group*  
*d rwx r-s r-x 2 root staff /srv/shared*

**sticky bit.** on directories, prevents deleting/renameing files you don't own.  
*d rwx rwx rwt 10 root tmp /tmp*

**Special Users.**

*root:* UID 0, full privileges, bypasses checks, in TCB.  
*sudo run as root su [username] switch to account(default is root)*  
*nobody:* UID -2, owns no files, minimal privileges, safer for untrusted code.(least privilege)  
⊗ flexible, simple model, widely adopted.  
⊗ relies on **ambient authority**, prone to Confused Deputy attacks.

**Windows: DACL.** Controls access to objects via list of ACEs.

**ACE (Access Control Entry).** <Type, Principal, Permissions, Flags>.

- **Types.** Allow / Deny (negative / positive). Deny takes precedence and ordered with Denied permissions first.
- **Principal.** User or group (SID).
- **Permissions.** Fine-grained rights(*Read, Write, Execute, Delete...*)
- **Flags.** Inheritance, propagation, object-specific.

**Access Tokens.** Thread/process carries user + group SIDs checked against DACL.

**Least Privilege.** Users run limited by default; elevation via “Run as admin.”

**Capabilities.** associate permissions to *subjects*

*Alice: {(file1, read/write)} Bob: {(file2, read/write), (file3, read/write)}*

⊗ easy to determine all permissions of a user and to delegate rights by subject.  
⊗ hard to determine who can access a resource or to revoke rights by resource.

**Subject Model.** a design pattern for a set of properties. (△ not covered by model - who are the subjects? what are the objects? what mechanisms to use to implement it?)

**Bell-La Padula (BLP).**  
security model where Subjects S and objects O are associated to a level of **Confidentiality**.  
**Access rights.** Execute, Read, Append, Write.  
- Objects are associated to a Security Level = (Classification, set of categories).  
*{{(Unclassified < Confidentiality < Secret < Top Secret), {NATO, Crypto, Nuclear}}}*  
**Dominance Relationship.**  
Transitive, There always is a Top and Bottom, Only partial ordering (some pairs of elements can't be compared).  
A security level ( $l_1, c_1$ . dominates ( $l_2, c_2$ . if and only if  $l_2 \leq l_1$  and  $c_2 \leq c_1$ .  
*eg. (Secret, {Nuclear, Army}) ≥ (Confidential, {Army})*  
**Clearance.** max security level a subject has been assigned. *clearance-level( $S_i$ )*.  
**Current Security level.** subjects can operate at lower security levels. *current-level( $S_i$ )*.  
**Declassification.** Removing classification labels from information so it can be shared with lower security levels. *soldier (S) with Secret clearance shares sanitized info like "We've seen tanks moving" with a lower-level user (U)*.

**Covert Channels.** Any channel that allows information flows contrary to the security policy, *Storage channels(shared counters, ...), Timing channels(queueing time. ...)*.  
△Least Common Mechanism. *mitigated by adding noise or isolation(no high → low level communication)*.

**BIBA (Integrity).**  
Security model where subjects S and objects O are 1. associated to a level of **Integrity** (trustworthiness).  
**Access rights.** Read (Observe), Write (Modify)  
**BIBA to create Integrity Policies.**  
**Simple Integrity Property.**  
If ( $s, o, r$ ) is a current access, then level(s) dominates level(o).  
**(SUBJECTS CAN'T READ DOWN)** ⇒ prevents contamination from low to high.  
**\*-Integrity Property.**  
If ( $s, o, w$ ) is a current access, then level(o) *does not* dominate level(s).  
**(SUBJECTS CAN'T WRITE UP)** ⇒ prevents low from corrupting high state.  
**Discretionary Property (DAC).** Subject must still have explicit permission for the access (r, w, x) per the access control matrix.  
**Basic Integrity Theorem (induction).** If the initial state is integrity-secure and all state transitions satisfy BIBA properties, then all reachable states preserve integrity.

**Sanitization (lifting low → high).**  
*Fail-safe default:* deny by default; elevate only after checks pass.  
- *Whitelist over blacklist:* validate that all required properties of "good" hold.  
- *Context-aware:* encoding, schema, range, semantics, and provenance checks.  
*Note:* Sanitization bugs commonly break integrity guarantees.  
**Principles for Integrity.**  
- **Separation of Duties.** Require multiple principals to perform an operation  
- **Rotation of Duties.** Allow a principal only a limited time on any particular role and limit other actions while in this role  
- **Secure Logging.** Tamper evident log to recover from integrity failures. Consistency of log across multiple entities is key.

**Chinese Wall Model.** All objects are associated with a label denoting their origin.  
*(Pepsi, Coca-Cola, Microsoft Audit, Microsoft Investments)*  
Define **conflict sets** of labels. *{Pepsi, Coca-Cola}, {Microsoft Audit, Microsoft Investments}*.  
Subjects are associated with a **history** of their accesses to objects and their labels.  
**Access Rules.** A subject can access an object (read or write) **only if** the access does not allow information flow between items with labels in the same conflict set.

**Example (Direct Flow).**  
1. Access to Pepsi (OK)  
2. Access to Microsoft Invest (OK)  
3. Access to Coca-Cola (Denied)  
**Example (Indirect Flow).**  
1. Alice accesses Pepsi  
2. Bob accesses Coca-Cola and IBM  
3. Alice tries to access IBM (Denied, indirect link via Bob)

**Sanitization.**  
Allows more flexibility by "un-labeling" some items→controlled sharing/reuse of data.

**All together (Asymmetric Cryptography) - Bob sends a message to Alice**  
**1. Get public keys (via PKI).**  
Bob retrieves Alice's encryption public key  $PK_{\text{Alice}}$  and verification public key  $PK_{\text{Alice}}$ .  
**2. Prepare the message.**  
Bob has message  $M$  and computes its hash  $h(M)$ .  
Allows: the signature to cover the exact content of  $M$  while keeping computation efficient.  
**3. Sign the message hash.**  
Bob uses his secret signing key  $SK_{\text{Bob}}$  to generate a digital signature.  $\text{Sign}_{SK_{\text{Bob}}}(h(M))$ .  
**4. Encrypt the message.**  
Bob encrypts the message using Alice's encryption public key  $PK_{\text{Alice}}$ :  $E_{PK_{\text{Alice}}}(M)$ .  
Allows: only Alice, who holds the matching secret key  $SK_{\text{Alice}}$ , to read the message.  
**Authenticity.** only sender can generate a valid signature, allowing the server to verify the sender's identity and preventing impersonation. **Integrity.** Any message change invalidates signature, ensuring tampering is detectable. **Non-repudiation.** only sender can produce the valid signature → they can't deny later having sent the message.

**Block Cipher - Electronic Code Block (ECB).**  
encrypt each message block independently using the same key.  
1. The message  $M$  is divided into blocks.  
 $M = M_1 M_2 M_3 M_4$ .  
2. Each block is encrypted separately  $C_i = E_K(M_i)$ .  
3. Decryption.  $M_i = D_K(C_i)$ .  
△If ( $M_i = M_j$ ) ⇒ ( $C_i = C_j$ ) ⇒ (freq. analysis)

**BLP to create Confidentiality Policies.**  
**Simple Security Property.**  
if (subject, object, w/r) is a current access, ⇒ level(subject) dominates level(object).  
**(SUBJECTS CAN'T READ UP)**  
**Star Property.** If a subject has simultaneous "observe" (r,w) access  $O_1$  and "alter" ( $a, w$ ) access to  $O_2$  then level  $O_2$  dominates level  $O_1$ .  
**(SUBJECTS CAN'T WRITE DOWN).** *changing object's perms is write-like.*

**Discretionary Property.** A subject may only access an object if it has explicit permission for that specific type of access (r, w, x, etc.) defined by the access control matrix.

**Basic Security Theorem(induction).** if all state transitions are secure, and the initial state is secure, then every subsequent state is secure regardless of the input.

**BIBA Low-Water-Mark Variants (taint-tracking).**  
**For Subjects.**  
Subjects start at their max integrity, **on read:**  $\text{current-level}(s) = \min(\text{current-level}(s), \text{level}(o))$ .  
⇒ once tainted, subject sinks; prevents writing up thereafter.  
**For Objects. On write by s:**  
 $\text{level}(o) = \min(\text{level}(o), \text{level}(s))$ .  
⇒ objects "sink" easily; can cause integrity collapse.  
*Mitigation:* replicate high/low objects; sanitize before promotion; detect/flag unexpected level drops.

**BIBA Additional Actions - Invoke.**  
**Simple Invocation.** only allow subjects to invoke subjects with a label they dominate.  
⊕ protect high integrity data from misues by low integrity principals  
⊖ what level is the output ?  
**Controlled Invocation.** Only allow subjects to invoke subjects that dominate them.  
⊕ prevents corruption of high integrity data  
⊖ hard to detect polluting information.

**Cryptography.**  
**Data in transit.** Securing communications. **Data at rest.** Securing stored informations  
Let C the Ciphertext, K the Key, M the Message/Plaintext.  $C = E_K(M)$  and  $M = D_K(C)$   
**Keyspace.** Number of possible keys that can be used in an encryption algorithm.  
**Invertibility Requirement.**  $\forall K, M | D_K(E_K(M)) = M$  (otherwise can't recover message)  
**Security Requirement.** Functions should be hard to invert without knowing K.  
Ideal Case - adversary must try every possible combination of keys (bruteforce).  
*Caesar's Cipher*  
- Encryption. Shift each letter by a fixed number (K)  
- Decryption. Shift each letter by a fixed number (-K)  
Keyspace. Only 25 possible keys.  
 $\log_2(25) = 4.6$  bits of security (too small).  
Both can be broken using **Frequency Analysis Attack**.

**Frequency Analysis.** Use statistical properties of the language.  
1. Most frequent letter in English is 'e'. 2. Identify most frequent letter in Ciphertext. 3. Map it to 'e'.  
**Ideally,** An N-bit key should offer security as close to N bits as possible (require  $2^N$  attempts), if not the algorithm is considered **broken**.  
**Types of Adversaries.** security models  
- Passive Eavesdropper - adversary can only read the ciphertext  
- Active Attacker - adversary can influence the system (*corruption of one of parties, ...*)  
**Known Plaintext Attack (KPA).** **Chosen Plaintext Attack (CPA).**  
Active security model. Attacker gets - Suppose Eve convinces Alice to encrypt chosen messages access to some pairs, (message, cipher- with the secret key text), all encrypted with secret K.  
- For all  $m$ , Eve gets access to  $E_K(m)$ . Eve has access to an **En-From these pairs, she tries to guess key** crypton Oracle.  
**K to decrypt other messages** (*realistic, she could get access to an encrypted messaging app realistic because of message headers, for limited time, or to an encryption api*).  
*message headers ("From:...", times-△if Eve encrypts entire alphabet she can reveal the key instantly).*

**Side-Channels Attacks**  
- Timing attack - Eve measure how long it takes for a given message to get encrypted or a ciphertext to be decrypted  
- Power Analysis - Eve observes the energy consumed by the device doing the crypto.

**One-Time-Pad (OTP).** OTP is **Perfectly Secure**.  
*Goal - Remove frequency analysis*  
- Use a key of random bits as along as the mes- equally likely  
*sage.*  
-  $\text{Enc}(k, m) = m \oplus k$   
-  $\text{Dec}(k, m) = \text{Enc}(k, m) \oplus k$   
Key should be random for every sent mes-  $\forall m, c, P(M = m | E_k(m) = c) = P(M = m)$ .  
*sage.* Otherwise attacker can collect informa- **Guarantees confidentiality**  
tion about the message. ⊕ message-length/used once keys.  
*Integrity Attack Example (Eve flips the first bit of M):* *Reuse of key Attack.* Send two messages of length 5 with key reuse.  
1. Eve flips the first of C to get C'  
 $C_1 \oplus C_2 = M_1 \oplus M_2$   
2. Bob decrypts C':  $M' = C' \oplus K = (M \oplus K \oplus \Delta) \oplus K = M \oplus \Delta$

**Symmetric Cryptography Schemes** **Symmetric Cryptographic key**  
Encryption/decryption done with the **-Known to both parties.** Partners must agree on the key before starting using the primitive  
**same key**  
- **Block Ciphers.** Operate on fixed-size blocks.  
- **Stream Ciphers.** Operate on bit/byte at a time, like pseudo-OTP.  
**-Reused.** The keys is pre-shared once and then reused (*but keys have a "duration"*)  
**-Must be secret.** Revealing the key eliminates any protection provided by the primitive

**Stream Ciphers - The pseudo-OTP.** emulate OTP while solving key-length problem.  
1. Secret short Key (K) shared between Alice and Bob 2. Key Stream Generator - Uses K and Initialization Vector (IV) → an arbitrary long, pseudo-random bit stream (S). 3. Encryption.  $C = M \oplus S$  (*generator needs a key as main seed to generate a predictable random sequence, an iv for the sequence to start differently on each run*)  
⊕ Speed, Low Error propagation(errors in one bit do not affect subsequent symbols)  
⊖ Low Diffusion (a change in a bit only affects one bit), Susceptibility to Modification (low diffusion makes it easier to tamper)  
⊖ key stream generators are periodic because seed is finite. Thus, we need period long enough to not an issue (avoiding frequency analysis)

**Linear Feedback Shift Register for Key Stream Generators.** build a ""random"" sequence of bits using a linear recurrence relation on a sequence of bit.  
*Example: Starting with state  $a_0, a_1, a_2, a_3 | a_n = a_{n-3} \oplus a_{n-4}$  (⊕ Easy to build/analyze.)*  
**Randomness of LFSR.** **Distribution property.**  
if characteristic polynomial of the recurrence relation of the LFSR is primitive. The maximum possible number of states before repeating. For an L-bit register, the maximum period is  $2^L - 1$  states  
As a consequence, the sequence generated exhibit good distribution properties. Every possible non-zero state appears exactly once in a cycle.  
⊖ underlying operation is **linear**. (*some algorithms can recover this with a stream subsequence*)

Symmetric ciphers are fast, but require a **pre-shared secret key**  
**Diffie-Hellman Setup.** Alice, Bob (and Eve) know large public parameters: prime p/generator g  
*Alice's Actions* *Bob's Actions*  
1. Chooses **private** secret a. 1. Chooses **private** secret b.  
2. Computes Public Value  $A = g^a \text{ mod } p$  2. Computes Public Value  $B = g^b \text{ mod } p$   
3. Sends A to Bob. 3. Sends B to Alice  
Eve sees A, B, g, and p, but can't find a/b due to the **Discrete Log Problem**.  
Both Alice and Bob can compute a shared secret K:  $K = B^a \text{ mod } p = (g^b)^a \text{ mod } p = A^b \text{ mod } p$  this is uses **Trapdoor functions**, easy to compute but hard to revert.

**Man-In-The-Middle (MITM).** 1. Eve sends  $E_B$  to Alice. Alice computes  $K_{AE}$   
Vanilla DH is Vulnerable to Active Attack, 2. Eve sends  $E_A$ . Bob computes  $K_{BE}$   
DH guarantees key agreement, **not** authenticity. 3. Creates an Alice-Eve channel and Eve-Bob channel  
**Attacker could intercept A from Alice and B she can read, modify, and relay all communication. from Bob, and then impersonate.**

**Asymetric Cryptography.** Encryption using a public-private key pair. Public key can be stored on a public server, the **Public Key Infrastructure (PKI)**.  
**Hash Functions.** maps any-length message to a fixed-size output using a hash function.  $h(M) = H$   
**Security properties.** 1. **Pre-image resistance.** Given  $H = h(M)$ , it is hard to find M.  
2. **Second pre-image resistance.** Given M, it is hard to find another  $M' \neq M$  such that  $h(M') = h(M)$ .  
3. **Collision resistance.** It is hard to find any two values M and M' such that  $h(M) = h(M')$ .

**Block Cipher - Counter Mode** **Block Cipher - CBC-MAC.** turns block cipher into MAC by chain-add randomness per-block without ing message blocks with encryption.  
inter-block dependencies.  $C_0 = IV$  **Encrypt.**  $C_i = E_K(M_i \oplus C_{i-1})$  MAC $_K(M_1, \dots, M_n) = C_n$   
(parallel) deterministic. fixed IV → same message/same MAC.  
**Encrypt.**  $C_i = M_i \oplus E_K(\text{Nonce} + i)$ . △message length must be known and fixed; otherwise, extension  
**Decrypt.**  $M_i = C_i \oplus E_K(\text{Nonce} + i)$ . attacks are possible (e.g., using  $M_i(T \oplus M')$ ).  
Nonce must be unique (never with same key) reuse leaks keystream.

**Message Authentication Code** phertext. 2. **MAC-then-Encrypt** send  $E_K(P_1 P_2 || MAC_K(P_1 P_2))$ . (MAC). short tag computed from hides MAC and provides confidentiality/integrity on plaintext but a message and secret key(pre-no integrity ciphertext.  
shared) to verify message **integrity** 3. **Encrypt-then-MAC** send  $E_K(P_1 P_2) || MAC_K(E_K(P_1 P_2))$ . **authenticates** ciphertext, ensures **confidentiality/integrity** ✓

**Authentication.** the process of verifying a claimed identity. (Prove who you are.)

**Passwords.** secret shared between user and system.

*Problems:*

- I **Secure transfer.** password may be eavesdropped when communicated.  
*Encrypting makes password unreadable to eavesdropper, but still allows copy and send.* ( $\Delta$ ) **Replay-Attacks.** sending same request without needing the password)
- Challenge-Response** protocol solves this.
  - 1. Principal declares they want to login.
  - 2. System authenticating principal sends a **Challenge** (a random value).
  - 3. Principal encrypts password *together* with the challenge.(ensures password is never encrypted with same value and encryption is fresh)
  - 4. After checking password, System *must* delete the challenge.
- II **Secure check.** naïve checks may leak information about the password.  
**Check Options:**
  - 1. Letter by letter - Time to check leaks the number of correct checked letters. (Timing channel)
  - 2. Check everything - Same time regardless of input (hashing solves this by default).
- III **Secure storage.** if stolen entire system is compromised.  
Password must be stored to be checked against but *not stored in clear* → an adversary can read them and impersonate users.  
**Storage Options :**
  - 1. Encrypted Passwords. requires key to be stored in same system as database, not secure.
  - 2. Hashed Passwords. Not a solution, hashing does **not** require a key.  $\Delta$ passwords are not truly random.→Database can be downloaded and compared against hash of most common passwords (**Dictionary Attack**).
  - 3. Hashed + Salted Passwords (Do this !). **Salt.** unique random value generated per password. **We store:** (username,  $H('123456'|salt_n), salt_n$ ).  
Dictionary attacks are still possible but same passwords are undetectable.  
*Principal* asks to login→*System* sends Challenge→*Principal* sends encrypted password with Challenge →*System* checks Challenge, decrypts, looks up hash by username, hashes plaintext with salt and checks if same as stored.
- IV **Secure passwords.** easy-to-remember passwords tend to be easy to guess

**Biometrics.** measurement and statistical analysis of people's unique physical characteristics(fingerprint, face, ...).

**Biometrics Authentication.**

- 1. **Enrollment.** biometric is captured by a sensor and processed to be converted in a stream of bits(Biometric Template). **Capture → Process → Store.**
- 2. **Verification.** biometric is captured, processed and compared against the stored one. **Capture → Process → Compare.** (threshold for match vs. no match).  
Balance between False Positives (wrongly authenticated) and False Negatives(wrongly rejected).  
Storing / Processing can happen locally(⊕ privacy) / remotely(⊕ security).

**Authentication Tokens.** by proving ownership of a token. (Smart Card, ...)

- 1. Offline - Device and Server establish a common *seed* (random value) & synchronize their clocks.
- 2. When Device wants identity:
  - (a) Server asks to prove it. (b) They both compute  $n = \frac{\text{now-start}}{\text{interval}}$ . *Current interval.*
  - (c) Device applies **keyed** function (pre-shared key) n times on initial seed.  $v = f^n(\text{seed})$
  - (d) Device sends result to Server. (encrypted, eavesdropper can't have the seed).
  - (e) Server computes same value and checks against the one received.

Can't be a hashing function (hash functions aren't keyed!), eavesdropper could compute  $h_k(\text{seed})$  for any k after the one observed.).

**Security Engineering process.**

- 1. Define security policy (*principals, assets, properties*) and a threat model.
- 2. Define security mechanisms that support policy of threat model.
- 3. Build implementation to support the mechanisms

**Threat Modelling.** process to identify threats / unprotected resources.

- **Attack Trees.** represent goal as the root and ways to achieve it as branches.
- **STRIDE.** 1) Identify assets, principals, trust boundaries, and named flows 2) **For each threat:**
  - 1. STRIDE category, 2.what changes/abuse happens, 3.which flow it affects, 4.concrete mitigation.
- Spoofing:** fake identity (user/service). **Tampering:** unauthorized edit of data/messages/code.
- Repudiation:** deny action (no provable record). **Info disclosure:** leak confidential data/metadata.
- Denial of service:** exhaust/block resources/reduce availability. **Elevation of privilege:** gain higher privileges than allowed.
- **PASTA.** start from buisness goals, processes, and use cases. Find threats in buisness model, assess impact, prioritize on risk.

**Methods.**

- **GET.** requests a resource; parameters are appended to the URL as query key–value pairs after "?", separated by '&'.

`http://www.mywebsite.com/apparel/skirt.php?sku=123&lang=en&sect=silk`

protocol

hostname / domain

directory

filename

query parameters

- **POST.** creates or updates a resource; sends data in the request body (not the URL) and typically modifies server data.

POST /test/demo\_form.php HTTP/1.1  
Host: w3schools.com  
name1=value1&name2=value2

**URL.** standardized address that specifies a resource's access method, host, and path.

**HTML.** markup language using tags to structure content and tell browser how to render a webpage.

**WEP.** Wi-Fi security protocol that encrypts frames with **RC4** (stream cipher with random 24-bit IV). IV repeats frequently, causing keystream reuse and enabling message (and sometimes key) recovery.

**PHP.** server-side scripting language for dynamic web pages; uses variables and request/session data ('\$\_GET', '\$\_POST', '\$\_SESSION') and outputs HTML with 'echo'.

*Lifetime of a GET/POST request.*

- 1. Browser sends HTTP GET/POST request.
- 2. Web Server receives requests and identifies if it's a static asset(.jpg,...) or dynamic(.php).
- 3. PHP interpreter executes called .php 4. Sends response to web browser

**Data Execution Prevention (DEP).** Mark memory pages as writable OR executable, never both. (prevents code injection by making stack/heap non-executable)

Virtual address space

Physical address space

⊕ low-overhead/hardware enforced  
⊖ no self-modifying code.

**Common Weakness Enumeration (CWE).** database of software errors leading to vulnerabilities.

- 1. **CWE 1 - Insecure Interaction Between Components.** unchecked information sent between different components.
  - (a) **CWE 78 - OS Command Injection.** application embeds unvalidated user input into an OS command, allowing attackers to execute arbitrary commands `ls /home/user`.
  - (b) **CWE 79 - Cross Site Scripting.** application embeds unvalidated user input into dynamically generated web content, causes browser to execute injected scripts.

**Common Script Injections.**

```
<img src=x onerror=alert(1)>;  
<svg onload=alert(1)>;  
<body onload=alert(1)>;  
<a href=javascript:alert(1)>text</a>;
```

**Useful Functions.**

|                                                                                                                             |                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>navigator.userAgent</code>                                                                                            | <code>document.domain</code>                                                                              |
| Returns the browser identification string; increases fingerprinting surface.                                                | Gets/sets legacy origin relaxation; can broaden trust across subdomains and weaken same-origin isolation. |
| <code>Chrome/143.0.0.0 Mobile Safari/537.36</code>                                                                          | <code>document.referrer</code>                                                                            |
| <code>window.location.replace(url)</code>                                                                                   | Returns the referrer URL when sent; can leak sensitive paths/parameters without strict Referrer-Policy.   |
| Forces navigation to url without adding a history entry.                                                                    | <code>document.URL</code>                                                                                 |
| <code>document.cookie</code>                                                                                                | Returns the full current document URL.                                                                    |
| Returns all non-HttpOnly cookies for the current origin; exposed session/state cookies immediate account compromise on XSS. |                                                                                                           |

- (c) **CWE 362 - Cross Site Request Forgery.** malicious website makes a state-changing action using user's session cookie for another website.

**Same Origin Policy (SOP).** web browser security mechanism that restricts scripts from one origin from reading data from another origin. An origin is a (protocol, host, port) `http:example.com`.  
*This doesn't prevent CSRF as it's based on sending a request and that the browser naturally attaches cookies of the destination domain in request (unless special setting).*

**Solutions:**

For every state-changing endpoint:

- Check Origin header and only allow exact origin.
- If Origin missing, require Referer and ensure it starts with expected origin, else reject.

*Keep GET side-effect free; Require a CSRF token (challenge) on every state-changing request; For high-riks actions require re-auth.*

- 2. **CWE 2 - Risky Ressource Management.** not properly managed creation, usage, transfer, or destruction of important system resources
- 3. **CWE 3 - Porous Defenses.** defensive techniques misused, abused, or just ignored

**Software Security.**

**Stack.** Call frames: locals, args, return addresses; grows down.

**Heap.** Dynamic allocations via `malloc/new`; grows up.

**Uninitialized data (BSS).** Global/static zero-initialized storage; zeroed at program start.

**Initialized data.** Global/static with explicit initial values; loaded from the executable.

**Text.** Executable code (often read-only); mapped from the executable.

**Args & env.** argv/environment strings placed at startup near the initial stack.

**Memory Corruption.**

Unintended modification of memory location due to missing / faulty safety checks.

```
void vuln(int user1, int*array){  
    // no bound check for user1  
    array[user1] = 42;}
```

```
void vulnerable(char* buf) {  
    free(buf); // variable was freed.  
    buf[12] = 42;}
```

**Memory Safety - Spatial Error.**

tried access to memory initially reserved for our program but not reserved anymore.

tried access to memory outside the bounds of the allocated region.

```
void vulnerable(){  
    char buf[12];  
    char *ptr = buf[11];  
    *ptr+=10;  
    // out of allocated memory  
    *ptr=42;}
```

```
int main(int argc, char** argv){  
    char buffer[100];  
    strncpy(buffer, argv[1]);  
    printf(buffer);  
    return 0;}
```

**Code Injection Attack.**

- 1. Control unchecked variable.
- 2. Overwrite return address.
- 3. Hijack control flow.
- 4. Arbitrary code execution.

```
void vuln(char* u1){  
    // strlen(u1) < MAX?  
    char tmp[MAX];  
    strcpy(tmp, u1);...  
} vuln(&exploit);
```

**Attack Chain:**

1. **Memory Safety Violation.** `strcpy` writes past `tmp[MAX]` (spatial error)
2. **Integrity (\*C).** Attacker overwrites return address on stack
3. **Location (&C).** Return address now points to shellcode
4. **Usage (\*&C).** CPU dereferences corrupted pointer
5. **Code Injection.** Shellcode executes with program privileges

**Address Space Layout Randomization (ASLR).**

Loader/OS randomizes memory regions(code/stack/heap/libraries) at load time. Same code, different addresses on each run. (prevents target addresses prediction/breaks gadgets)

**Limitations:**

- 1. Probabilistic defense, not deterministic.
- 2. Vulnerable to addresses leaks.
- 3. Some regions remain static *x86*.
- 4. Performance overhead.

**Stack Canaries.**

Compiler places random secret value between local buffers and return address. Checked before function returns and abort if corrupted. (*detects buffer overflows before control hijack*)

**Limitations:**

- 1. Probabilistic, vulnerable to canary leaks.
- 2. No protection against targeted writes (jumps over canary).
- 3. No protection against reads.
- 4. Performance overhead from checks.

**Testing.** process of executing a program to find errors. **Error.** deviation between observed behavior and specified behavior (a violation of the underlying specification)